

DISCOVERING AZURE MACHINE LEARNING

How to Efficiently Use Azure AI for Deploying Machine Learning Models

A Practical Field Guide for Aspiring Data Scientists & ML Engineers

SECTION 1 — THEORY: Understanding Azure Machine Learning

Chapter 1: Introduction to Azure Machine Learning

Azure Machine Learning (Azure ML) is Microsoft's comprehensive, cloud-based platform purpose-built to streamline the entire machine learning lifecycle — from raw data ingestion all the way through to model deployment and monitoring in production. Whether you are building classical ML models or fine-tuning large language models (LLMs), Azure ML provides the infrastructure, tooling, and integrations to do so at enterprise scale.

This chapter covers the foundational concepts you need to understand before working with Azure ML hands-on. Think of it as building the mental model that will make every subsequent practical step intuitive and meaningful.

1.1 What Is Azure Machine Learning?

Azure ML is not a standalone product — it is part of the broader Microsoft Azure ecosystem, which means it benefits from tight integration with other Azure services such as Azure Blob Storage, Azure Data Factory, Azure Kubernetes Service (AKS), Azure SQL Database, and Azure Active Directory. This integration is one of its most powerful characteristics.

Key Definition

Azure Machine Learning is a cloud service for accelerating and managing the machine learning project lifecycle.

It enables data scientists, ML engineers, and developers to train, deploy, manage, and track ML models in a production-grade environment without managing underlying infrastructure.

At its core, Azure ML provides:

- A unified workspace to manage all ML assets and compute resources
- Support for both code-first and low-code/no-code workflows
- Scalable compute infrastructure (CPU, GPU, and distributed clusters)
- A rich Model Catalog for leveraging pre-built and pre-trained models
- End-to-end MLOps capabilities for production deployment and monitoring

1.2 The Azure ML Workspace — Your Headquarters

The Azure ML Workspace is the central organizing unit in Azure ML. Think of it as the headquarters of your entire ML operation. Every asset, every compute resource, every trained model, and every deployment lives under and is managed by a workspace.

When you create an Azure ML Workspace, Azure automatically provisions and links several supporting resources:

Resource	Purpose
Azure Storage Account	Stores datasets, model artifacts, training scripts, and experiment outputs
Azure Key Vault	Securely manages credentials, API keys, and connection strings used by the workspace
Azure Application Insights	Collects telemetry data to monitor deployed model endpoints
Azure Container Registry	Stores Docker images used to create consistent training and inference environments

Together, these resources ensure that your workspace is secure, observable, and reproducible. You rarely interact with them directly — Azure ML abstracts them away — but it is important to know they exist because they appear in your Azure subscription and contribute to costs.

1.3 Compute in Azure ML

One of the most important decisions when working with Azure ML is choosing the right compute. Azure ML offers several compute types, each suited to different stages of the ML workflow:

Compute Type	Best Used For
Compute Instance	A managed virtual machine used as your personal development environment — ideal for running Jupyter Notebooks, experiments, and debugging
Compute Cluster	A scalable pool of nodes (CPU or GPU) that auto-scales up during training and scales down to zero when idle — cost-efficient for batch training jobs

Compute Type	Best Used For
Inference Cluster (AKS)	Azure Kubernetes Service cluster used for deploying models as real-time, high-throughput REST API endpoints
Attached Compute	Existing Azure VMs, Databricks clusters, or HDInsight clusters that you attach to your workspace for specialized workloads

Cost Tip

Compute Clusters are billed only when running. Always configure 'minimum nodes = 0' so the cluster scales to zero when not in use.

GPU clusters (e.g., Standard_NC12s_v3 with 2x NVIDIA Tesla V100) are powerful but expensive at ~\$1.22/hr. Use them only for training jobs that genuinely require GPU acceleration.

1.4 Data Management in Azure ML

Data is the fuel of any ML system. Azure ML provides a structured way to manage data through two core concepts: Datastores and Data Assets (Datasets).

Datastores

A Datastore is a reference (connection) to an external storage service. It does not copy or move your data — it stores credentials and connection information so Azure ML jobs can securely access data at runtime. Supported datastores include Azure Blob Storage, Azure Data Lake Storage Gen1/Gen2, Azure SQL Database, and Azure File Shares.

Data Assets

A Data Asset is a versioned reference to a specific dataset stored in a datastore. Think of it as a named, versioned pointer to your data. When you register a CSV file as a Data Asset, you can reference it by name and version across training jobs, ensuring reproducibility.

The typical data flow in an Azure ML project looks like this:

1. Raw data (CSV, Parquet, JSON, etc.) is ingested via Azure Data Factory or uploaded directly
2. Data is stored in Azure Blob Storage or Azure Data Lake Storage
3. A Datastore connection is created in the Azure ML workspace
4. A Data Asset is registered pointing to the specific file/folder
5. Training scripts reference the Data Asset by name — ensuring consistency and auditability

1.5 Building and Deploying Models — The End-to-End Pipeline

Azure ML supports the complete ML model lifecycle. Here is how the stages connect:

Stage	What Happens
1. Ingest	Raw data (e.g., CSV) is pulled from source systems using Azure Data Factory or uploaded directly to the workspace
2. Store	Data lands in Azure Data Lake Storage or Blob Storage, registered as a Data Asset in the workspace
3. Train	Azure ML jobs run on compute clusters — using Automated ML, Designer (drag-and-drop), or custom Python scripts in Notebooks
4. Deploy	Trained models are registered in the Model Registry and deployed as real-time endpoints (ACI/AKS) or batch endpoints
5. Consume	Downstream applications (Power Apps, Power Automate, custom APIs) call the deployed model endpoint for predictions

1.6 The Model Catalog — Standing on the Shoulders of Giants

One of the most powerful features of Azure ML is the Model Catalog — a rich, curated repository of pre-trained models from Microsoft, Hugging Face, Meta, Mistral AI, and other leading research organizations. Instead of training a model from scratch (which requires massive datasets, compute, and time), the Model Catalog lets you:

- Discover models organized by task type — text classification, text generation, image classification, object detection, and more
- Evaluate a model's performance on benchmark datasets before committing to it
- Fine-tune a pre-trained model on your own domain-specific data with just a few clicks or a few lines of code
- Deploy a model directly to a real-time endpoint from the catalog UI

Pre-training vs. Fine-tuning — Understanding the Difference

This distinction is critical for any practitioner working with modern ML, especially LLMs:

Concept	Description
Pre-training	Training a model from scratch on massive general datasets (e.g., billions of tokens of text). Done by organizations like Meta, Google, or Mistral. Extremely compute and time intensive. You rarely do this yourself.
Fine-tuning	Taking a pre-trained base model and continuing its training on a smaller, task-specific dataset. This adapts the model's learned representations to your specific domain, labels, or output format. This is what you typically do in practice.

Why Fine-tune Instead of Training from Scratch?
A pre-trained model like T5-base has already learned language structure, grammar, and world knowledge from 800GB+ of text data.

Fine-tuning on 10,000 domain-specific labeled examples takes hours instead of weeks and requires far less compute.

This transfer learning paradigm is the foundation of modern applied ML — it is why small teams can build powerful models.

1.7 Deployment Options

Once a model is trained and registered, Azure ML offers three primary deployment pathways:

Deployment Type	When to Use It
Real-time Endpoint	For low-latency, synchronous predictions — e.g., a user submits text and gets an immediate sentiment score. Backed by ACI (for dev/test) or AKS (for production).
Batch Endpoint	For large-scale, asynchronous scoring jobs — e.g., running inference on 1 million records overnight. Cost-efficient as compute only runs during the job.
Web Service (Framework)	For deploying models built with specific frameworks (e.g., TensorFlow Serving, ONNX Runtime) that expose predictions via REST API.

Architecture Insight

Real-time endpoints can be deployed on Azure Container Instances (ACI) for quick dev/test setups or Azure Kubernetes Service (AKS) for production-grade, auto-scaling deployments. Once deployed, the endpoint exposes a REST API with a scoring URI and an authentication key. Any application that can make an HTTP POST request can consume your model.

SECTION 2 — PRACTICAL: Building, Fine-Tuning & Deploying a Model in Azure ML

Chapter 2: Hands-On — Fine-Tuning an LLM for Sentiment Analysis

In this practical walkthrough, you will go from zero to a deployed, callable sentiment analysis model using Azure ML Studio. The model we fine-tune is T5-base — a Text-to-Text Transfer Transformer developed by Google that reframes every NLP task (classification, translation, summarization) as a text generation problem. We fine-tune it on a Yelp review dataset to predict star ratings from review text.

What You Will Build

A fine-tuned T5-base model trained on Yelp review data (text -> star rating classification)

A deployed real-time inference endpoint callable via REST API

The complete Azure ML setup: workspace, compute cluster, data asset, training job, model registration, and endpoint

Step 1 — Create an Azure ML Workspace

1.1 Open the Azure Portal

Navigate to portal.azure.com and sign in with your Azure account. In the top search bar, type 'Azure Machine Learning' and select it from the services list.

1.2 Create a New Resource

Click '+ Create' to begin the workspace creation wizard. You will be asked to fill in the following configuration:

Field	What to Enter / Why It Matters
Subscription	Select your active Azure subscription — this is where billing is tracked
Resource Group	Create a new one (e.g., 'azure-ml-rg') — a resource group logically groups all related Azure resources together
Workspace Name	Give it a meaningful name (e.g., 'ml-workspace-prod') — this becomes the identifier you use throughout the project
Region	Choose the Azure region closest to your users/data for lower latency and potential cost savings (e.g., West US 2, East US)
Storage Account	Auto-created — stores all workspace artifacts, datasets, and model outputs
Key Vault	Auto-created — securely stores credentials and secrets used by the workspace
Application Insights	Auto-created — enables monitoring and telemetry for your deployed endpoints
Container Registry	Can be set to None initially and created later when you need custom Docker environments

1.3 Review + Create

Click through the Networking, Encryption, and Identity tabs (defaults are fine for a learning environment). On the final 'Review + Create' tab, Azure validates your configuration. Click 'Create'. Provisioning typically takes 2-4 minutes.

1.4 Launch Azure ML Studio

Once provisioning is complete, navigate to your new resource and click 'Launch Studio'. This opens Azure ML Studio — the web-based IDE for your entire ML workflow. The home dashboard shows quick-access shortcuts for: Add compute, Connect data, Train a model, and Deploy a model.

Step 2 — Prepare Your Training Data

2.1 Understanding the Dataset Format

For fine-tuning T5-base on a text classification task, your training CSV must contain at minimum two columns:

- sentence1 — the text input (the review text)
- label — the target output (the star rating as an integer: 1-5)

The Yelp dataset used in this example has columns: business_id, date, review_id, stars, text, type, user_id, cool, useful, funny. We map 'text' to sentence1 and 'stars' to label.

2.2 Uploading the CSV File

In Azure ML Studio, navigate to Data (left sidebar) > + Create. Choose 'From local files' and upload your CSV. Azure ML will automatically profile the data and show a preview. Give the dataset a name (e.g., 'yelp_reviews') and version (1.0).

2.3 Mapping Columns

The fine-tuning wizard will prompt you to map your dataset columns to the expected schema. This is a critical step:

Schema Field	Maps To (in our dataset)
sentence1 key	text (the review text — this is the model input)
label key	stars (the star rating — this is what the model learns to predict)
sentence2 key (optional)	Not required for single-sentence classification

Step 3 — Configure and Launch Fine-Tuning

3.1 Navigate to the Model Catalog

From the Azure ML Studio left sidebar, click 'Model catalog'. You will see hundreds of models grouped by publisher and task. Use the filter panel on the right to filter by 'Fine-tuning tasks: Text classification'. You will see models like T5-small, T5-base, T5-large, roberta-base, roberta-large, mistralai-Mistral-7B, and others.

3.2 Select T5-base

Click on 't5-base'. This opens the model detail page which shows the model card, license information (Apache 2.0), supported languages (en, fr, ro, de), and available actions: Refresh, Evaluate, Fine-tune, Deploy. Click 'Fine-tune'.

Why T5-base?

T5 (Text-to-Text Transfer Transformer) treats every NLP task as a sequence-to-sequence problem.

T5-base has 220 million parameters — powerful enough for quality results, yet manageable enough to fine-tune on an Azure GPU cluster within a reasonable time and cost.

The Apache 2.0 license means you can use it commercially without restrictions.

3.3 Fine-Tune Configuration

In the 'Fine-tune t5-base' wizard, configure the following:

Setting	Value & Explanation
Select task type	Text classification — tells Azure ML to set up the training pipeline for classification (softmax head over the encoder output)
Training data	Select your uploaded Yelp dataset — the data the model will learn from
Validation data	Automatic split of training data — Azure ML will hold out ~10% of your training data for validation
Test data	Automatic split of training data — used to compute final evaluation metrics after training completes
Azure ML compute cluster	Create new (see Step 4) — the GPU cluster that will run the training job

Step 4 — Create a GPU Compute Cluster

4.1 Why a GPU Cluster?

Fine-tuning a 220M parameter model like T5-base on thousands of examples requires significant matrix multiplication operations. GPUs are purpose-built for this — a job that takes hours on a CPU cluster completes in under an hour on a GPU cluster. Azure ML makes it easy to provision GPU clusters on demand.

4.2 Cluster Configuration

In the 'Select Azure ML compute cluster' step, click '+ New'. The cluster wizard has two pages:

Page 1 — Virtual Machine Selection

Choose GPU as the virtual machine type. For fine-tuning T5-base, the recommended selection is Standard_NC12s_v3:

VM Size	GPUs	RAM	Storage	Cost/Node
Standard_NC6s_v3	1x NVIDIA Tesla V100 (16GB)	112 GB	736 GB	~\$0.61/hr

VM Size	GPUs	RAM	Storage	Cost/Node
Standard_NC12s_v3 (Recommended)	2x NVIDIA Tesla V100 (16GB ea.)	224 GB	1,474 GB	~\$1.22/hr
Standard_NC24s_v3	4x NVIDIA Tesla V100 (16GB ea.)	448 GB	2,948 GB	~\$2.45/hr
Standard_ND40rs_v2	8x NVIDIA Tesla V100 (32GB ea.)	672 GB	2,900 GB	~\$4.41/hr

Page 2 — Cluster Settings

Setting	Recommended Value & Reason
Compute name	Auto-generated or custom name (e.g., 'gpu-cluster-t5')
Minimum number of nodes	0 — cluster scales to zero when idle, eliminating standby costs
Maximum number of nodes	1 — for a single fine-tuning job; increase for parallel hyperparameter sweeps
Idle seconds before scale down	30 seconds — cluster deallocates quickly after job completion to save cost
Enable SSH access	Optional — enable only if you need to debug inside the cluster nodes

Important: Training Time Expectation

Fine-tuning T5-base on a Yelp-sized dataset with a Standard_NC12s_v3 cluster takes more than 1 hour to complete.

During training, Azure ML logs metrics (loss, accuracy, F1 score) in real-time to the Jobs section.

You do not need to keep the browser open — the job runs server-side. You can check progress anytime.

Step 5 — Monitor the Training Job

After clicking 'Finish' in the fine-tuning wizard, Azure ML submits a training Job. Navigate to Jobs in the left sidebar to find your job. The job page shows:

- Status: Preparing → Queued → Running → Completed
- Real-time metrics: training loss, validation loss, accuracy plotted over epochs
- Logs: console output from the training script, useful for debugging
- Outputs: all artifacts produced by the job, including the trained model checkpoint

Step 6 — Register the Trained Model

6.1 Why Model Registration?

Model registration in Azure ML creates a versioned, named entry in the Model Registry for your trained model. This is critical for MLOps — it allows you to track which version is deployed, roll back to previous versions, and audit the lineage (which data and compute were used to produce this model).

6.2 Register from Job Output

Once the training job completes, navigate to its Outputs tab. Click 'Register model'. In the registration wizard:

6. Select job: Your completed fine-tuning job
7. Select output: Azure ML auto-detects the named model output and pre-selects it
8. Model type: MLflow — T5 fine-tuning jobs produce MLflow-compatible models by default
9. Job output: Displays the full artifact path (e.g., 'trained_model (azureml_6c8d0c77-86f3-4132-86c0-bff16c2b4f48_output_mlf...')
10. Click Next and provide a name (e.g., 't5-sentiment-v1') and optional description

Step 7 — Deploy the Model as a Real-Time Endpoint

7.1 Initiate Deployment

From the registered model's detail page, click 'Deploy' and select 'Real-time endpoint'. This opens the deployment wizard which will provision the inference infrastructure.

7.2 Select a Virtual Machine

For a real-time inference endpoint, you do not need a GPU (inference is much cheaper than training). A CPU-based VM works well for T5-base inference:

VM Size	Specs	Cost/Hr	Best For
Standard_DS3_v2 (Default)	4 cores, 14 GB RAM, 28 GB Disk	\$0.29/hr	Development and testing endpoints
Standard_F4s_v2	4 cores, 8 GB RAM, 32 GB storage	\$0.17/hr	Cost-optimized inference
Standard_F8s_v2	8 cores, 16 GB RAM, 64 GB storage	\$0.34/hr	Higher throughput endpoints
Standard_NC12s_v3	12 cores, 2x V100 GPUs	\$6.12/hr	Low-latency GPU inference for large models

7.3 Set Instance Count

Set instance count to 3 for a production-grade endpoint. This means Azure deploys your model on 3 virtual machine instances simultaneously, providing:

- High availability — if one instance fails, the other two continue serving traffic

- Load balancing — requests are distributed across instances for better throughput
- Zero-downtime rolling updates — when you redeploy with a new model version

7.4 Deploy

Click 'Deploy'. Azure provisions the infrastructure, builds a Docker container with your model and dependencies, and deploys it across your 3 instances. This process takes approximately 10 minutes. Once complete, the endpoint status changes to 'Healthy'.

Step 8 — Test the Deployed Endpoint

8.1 Using the Test Tab in Azure ML Studio

Navigate to Endpoints in the left sidebar, click your endpoint, and go to the 'Test' tab. Azure ML provides a built-in inference UI where you can send test requests. The input format for the T5 text classification endpoint is:

```
{
  "input_data": [
    "This is the worst restaurant I have ever been to!"
  ],
  "params": {
    "return_all_scores": true
  }
}
```

8.2 Understanding the Response

The endpoint returns a JSON array with label-score pairs for each possible class. For a 5-star rating model, you will see scores for labels 1 through 5:

```
[
  [
    { "0": { "label": 1, "score": 0.09499499492168426 } },
    { "1": { "label": 2, "score": 0.13088606048078537 } },
    { "2": { "label": 3, "score": 0.19234512 } },
    { "3": { "label": 4, "score": 0.26802128 } },
    { "4": { "label": 5, "score": 0.31375254 } }
  ]
]
```

In this example, the model assigns the highest score to label 5, but since the input text is 'This is the worst restaurant I have ever been to!' — a clearly negative review — a well-trained model would assign high probability to label 1. If your model's predictions don't align with expectations, it may need more training data, more epochs, or better hyperparameter tuning.

8.3 Calling the Endpoint via Python (Programmatic Access)

In real-world applications, you will call the endpoint from your own Python code. Here is a complete, ready-to-run example:

```

# Cell 1 – Install required libraries
# Run this in a terminal or a notebook cell
# pip install azure-ai-ml azure-identity requests

# -----
# Cell 2 – Import Libraries
# -----
import json
import requests
from azure.ai.ml import MLClient
from azure.identity import DefaultAzureCredential

# Explanation:
# MLClient is the main SDK client for interacting with Azure ML.
# DefaultAzureCredential handles authentication automatically –
# it checks environment variables, managed identity, and your
# local Azure CLI login in sequence.

# -----
# Cell 3 – Authenticate and Connect to the Workspace
# -----
credential = DefaultAzureCredential()

ml_client = MLClient(
    credential=credential,
    subscription_id="",
    resource_group_name="",
    workspace_name=""
)

# Explanation:
# MLClient establishes a connection to your specific Azure ML workspace.
# Replace the placeholder values with your actual subscription ID,
# resource group name, and workspace name from the Azure portal.

# -----
# Cell 4 – Retrieve the Endpoint Details
# -----
endpoint_name = "t5-sentiment-endpoint"
endpoint = ml_client.online_endpoints.get(name=endpoint_name)
scoring_uri = endpoint.scoring_uri

# Get the primary authentication key
keys = ml_client.online_endpoints.get_keys(name=endpoint_name)
api_key = keys.primary_key

print(f'Scoring URI: {scoring_uri}')
print(f'API Key: {api_key[:10]}...')

# Explanation:
# Every deployed endpoint has a scoring_uri (the REST API URL) and
# an authentication key. You need both to make authorized requests.
# The key should be treated like a password – never hardcode it
# in source code. Use environment variables or Azure Key Vault.

```

```

# -----
# Cell 5 – Send Inference Requests
# -----
headers = {
    "Content-Type": "application/json",
    "Authorization": f"Bearer {api_key}"
}

payload = {
    "input_data": [
        "This is the worst restaurant I have ever been to!",
        "Amazing food, great service, will definitely come back!"
    ],
    "params": {
        "return_all_scores": True
    }
}

response = requests.post(scoring_uri, headers=headers, json=payload)

# Explanation:
# We send a POST request to the endpoint's scoring URI.
# The payload contains a batch of two review texts – the endpoint
# will return classification scores for each input.
# 'return_all_scores: True' returns probabilities for all classes
# instead of just the top predicted class.

# -----
# Cell 6 – Parse and Display Results
# -----
if response.status_code == 200:
    results = response.json()
    for i, review_scores in enumerate(results):
        print(f"\n--- Review {i+1} ---")
        for label_id, score_data in review_scores[0].items():
            label = score_data["label"]
            score = score_data["score"]
            print(f"  Star Rating {label}: {score:.4f} ({score*100:.1f}%)")
else:
    print(f"Error: {response.status_code}")
    print(response.text)

# Explanation:
# We parse the JSON response and display the confidence score
# for each star rating (1-5) for each input review.
# The label with the highest score is the model's prediction.
# A well-trained model should give negative reviews high scores
# on label 1 and positive reviews high scores on label 4 or 5.

```

Summary: The Complete Azure ML Workflow

Step	Action & Key Takeaway
1. Create Workspace	Your Azure ML headquarters — all assets, compute, and models live here. Supporting resources (Storage, Key Vault, App Insights) are auto-provisioned.
2. Prepare Data	Upload your CSV and register it as a versioned Data Asset. Map columns to the expected schema (sentence1 = input text, label = target class).
3. Configure Fine-Tuning	Select T5-base from the Model Catalog. Set task = Text Classification. Link your training data. Configure validation as auto-split.
4. Create GPU Cluster	Standard_NC12s_v3 (2x V100 GPUs) is recommended. Set min nodes = 0 to avoid standby costs. Training takes 1+ hour.
5. Monitor Job	Check real-time metrics in the Jobs panel. Azure ML logs loss and accuracy per epoch. Job runs server-side — no need to stay online.
6. Register Model	Register the trained model from job output as an MLflow model with a version tag. This creates a deployable, auditable artifact.
7. Deploy Endpoint	Deploy as a Real-time endpoint on 3 CPU instances (Standard_DS3_v2). Deployment takes ~10 minutes. Endpoint becomes a REST API.
8. Test & Integrate	Test via the Studio UI or call programmatically via Python using azure-ai-ml SDK. Pass review text, receive star-rating probabilities.

Next Steps — Where to Go From Here

Experiment with hyperparameters: In the fine-tuning wizard's Advanced Settings, try adjusting learning rate, batch size, and number of epochs to improve accuracy.

Try other models: The Model Catalog includes Phi-3-mini, Llama-2-7b, and Mistral-7B for more powerful text understanding tasks.

Build a Prompt Flow: Azure ML's Prompt Flow feature lets you chain LLM calls, data lookups, and Python functions into a visual pipeline for complex GenAI applications.

Set up MLOps: Explore Azure ML Pipelines to automate retraining, A/B testing of model versions, and automated deployment on data drift detection.